

QuizRx Beta v1.0

Shared Request / Response Contract — Frontend ⇄ Cognitive Service

The single interface both tracks build against (Spec Part III, X-01). The frontend sends one request shape for every learning action; the backend always answers with a typed response — the frontend renders the type and never assumes a question card. Agree this page first; everything else plugs into it.

1 Request — every learning-action call carries

Field	Presence	Meaning
message	REQUIRED	The user's raw message, or the selected suggested-prompt intent (e.g. <code>generate_question</code>).
topic_id	REQUIRED	Selected topic id, or explicit <code>null</code> for the None state — never omitted, so the backend can apply the no-topic rules.
topic_display_name	OPTIONAL	Display name as shown to the user (for approved copy that echoes the topic).
session_id	REQUIRED	Current session id — history and feedback join on it.
current_question_id	OPTIONAL	The question currently on screen. Required whenever the user submits a follow-up or review (e.g. "why is option B wrong?") so the backend can resolve option references.
user_id	OPTIONAL	Authenticated user id when available.

2 Response — always one of seven types

Every response carries `response_type` plus a payload. The frontend switches on the type — it never guesses from payload shape, and never renders a question card unless the type says so.

small_talk — approved greeting/social reply. Payload: `text`.

scope_redirect — outside-scope message with the approved redirect copy. Payload: `text`.

explanation — concise explanation with the approved closing invitation. Payload: `text`. Never followed by an auto-generated question.

question — render the question card. Payload: the **question object** (Section 3).

question_review — expanded explanation of the current/identified question. Payload: `text` + `question_id`.

clarification — the backend asks one clarifying question for an ambiguous request (asked once, never looped). Payload: `text`.

friendly_error — approved user-facing failure copy, e.g. "Sorry, I couldn't generate a question just now. Please try again." Payload: `text`.

3 The question object is source-neutral

QuizRx will run two question systems — curated (QuizRx Reasoning) and AI-generated (Practice Studio). The card must render both identically, so the object carries a generic id and a source tag; nothing in the frontend may key on pipeline-specific ids.

Field	Meaning
question_id	Generic unique id. The frontend echoes it back in follow-ups (<code>current_question_id</code>) and reports — it never parses or interprets it.
source	<code>generated</code> or <code>curated</code> — informational only; rendering is identical.
stem, options[]	Question text and options (<code>label</code> , <code>text</code>). 3–5 options; the card renders however many arrive.
correct_label	For grading the user's pick.
explanation	Fixed order: correct answer → why it's correct → why each other option is wrong. No source/trace line.
topic_id	The topic this question belongs to.

Reporting a question ("Report this question")

POST `/endocrinology/question-feedback/` with `question_id` , `reason` (required, exactly one of the 5 approved values below), optional `free_text` comment, plus `session_id` / `user_id` . The thumbs rating is retired.

`incorrect_answer` · `explanation_unclear` · `question_unclear` · `typo` · `other`

BACKEND OWNS ROUTING

Message classification and intent mapping happen server-side. The frontend sends the message and renders the typed response — it performs no local small-talk detection once this contract is live.

EXACT COPY ONLY

All user-facing strings in typed responses come from the approved copy set. The frontend never rewrites, truncates or substitutes them.

CHANGE CONTROL

Any field addition or type change to this contract is agreed by both tracks before either side ships it.